Per Scholas

Report

CAP 931

Sales Agent Prototype: Sales Account Research Assistant

Richard Watson Jr.

Resubmitted October 22, 2026 (first submitted October 16, 2026)

Live app: https://cap931-sales-agent-rwjr.streamlit.app/
Code: https://github.com/rwnycvet/cap931-sales-agent

Per Scholas

Resubmission Changes **2**
Overview **3**
Steps **4**
Conclusion **13**
Appendix **14**
A. Time Management 14
B. Challenges and Solutions 14
C. Experiments Log 16
D. Final Prompts 17
E. Code Structure 18
F. System Output: Sample One-Pager 19
G. Test Case, Sources and AI Assistance 20

Per Scholas

Resubmission Changes

This resubmission responds to the instructor review of October 21, 2026 (114/140). Every item below is in the GitHub repository; the main point of the review was that the report and presentation were not in the repo, so the evidence for several rubric areas could not be seen.

Review feedback	What I changed	Where
Report and presentation not in the repository	Added the final report, presentation and a sample one-pager to a `docs/` folder	`docs/`
README empty; placeholder project description	Wrote a full README (purpose, architecture, setup, tests, deployment, limitations, security); real `pyproject.toml` description	`README.md`
No automated tests	Moved all logic into `agent_core.py` and added 42 pytest tests covering every area the review listed	Step 11, `tests/`
URL validation and SSRF risk	`is_safe_url()` allows only http/https and blocks private, loopback, link-local and reserved IP addresses; pages over 2 MB are refused	Steps 2 and 5
Competitor research limited to name checks	The app now downloads each competitor's homepage and summarizes its positioning from that page only	Step 7
Pages had to be entered by hand	Auto-discovery finds About, Leadership, News, Careers and Investor pages from the homepage links	Step 6
No controlled comparison of prompt designs or models	Ran a controlled experiment: 3-step chain vs. single prompt, and gpt-oss-120b vs. gpt-oss-20b, on identical saved evidence	Step 12, Appendix C
Leadership accuracy check not completed	Checked against MCU's leadership page: 5/5 in the report sample, 6/6 in the repo sample	Appendix F
Production plan too brief	Expanded to cover login, HTTPS, rate limiting, caching, queues, monitoring, logs, cost budgets, CI/CD, rollback, data retention and user isolation	Step 13

What I did not build: the email alert system remains a design (Step 14). I prioritized the items the review weighted most.

Overview

The goal of this capstone was to build a prototype sales assistant agent that helps a new sales representative prepare for a first meeting with a prospective account. The rep enters a product, a target company, competitors, a value proposition and the person they are meeting. The agent reads the company's public web pages and an optional product sheet, and returns a one-page account brief covering company strategy, leadership, competitor mentions, likely needs, talking points and source links.

I built the agent in Python with a Streamlit interface and OpenAI's open-weight GPT model **gpt-oss-120b**, served through the Groq API. The final design is a **three-step prompt chain**: the model first extracts only the facts stated on the company's pages, then identifies likely needs from those facts alone, and finally writes the brief. Plain Python handles every task that has an exact answer: input validation, URL safety checks, finding and downloading web pages, reading the product PDF, checking for competitor mentions and listing sources. All of that logic lives in `agent_core.py`, which has no interface code, so it is covered by 42 automated tests.

I tested eight prompt versions against one fixed test case (a fictional endpoint security product, GreatShield EDR, sold to Municipal Credit Union in New York City). The experiments showed that rules in a prompt reduce hallucination, but only real data adds facts, and that structural changes to what the model can see fixed problems that additional rules could not.

What was delivered

- A working Streamlit app with all required inputs, input validation and error handling
- URL safety checks that block private and internal addresses (SSRF protection) and oversized pages
- Web data integration from the company homepage, pages entered by the rep, and pages found automatically (About, Leadership, News, Careers, Investor)
- A Python competitor-mention check, plus research of each competitor's own website
- Product overview PDF parsing
- A three-step chained prompt pipeline with visible intermediate results
- A downloadable one-page account brief with sources
- A pytest suite of 42 automated tests and a controlled experiment comparing prompt designs and models
- Public deployment on Streamlit Community Cloud, with the API key stored in platform secrets
- A written design for an email alert system and a production deployment plan

Steps

The steps below follow the order in which I built the system. Steps 1–10 describe the original build; Steps 11–12 and parts of Steps 2, 5, 6, 7 and 13 were added for this resubmission. Each step lists what I did, why, and how I verified it. Detailed problems and fixes are in Appendix B, and the full experiment log is in Appendix C.

Step 1. Set up the development environment

I worked on Windows in Visual Studio Code, using **uv** to manage Python and dependencies and **Git** for version control. I kept the API key in a `.env` file and added `.env` to `.gitignore` before creating it, so the key could never be committed. Before publishing to GitHub I confirmed with `git ls-files` that `.env` was not tracked.

To reproduce the environment:

```
git clone https://github.com/rwnycvet/cap931-sales-agent.git
cd cap931-sales-agent
uv sync                      # or: pip install -r requirements.txt
echo GROQ_API_KEY=your-key > .env
uv run streamlit run app.py
```

Libraries used: streamlit (interface), groq (model API), python-dotenv (loads the key), requests and beautifulsoup4 (download and clean web pages), pypdf (read the product sheet).

Step 2. Build the input form and validate inputs

The Streamlit form collects every input the assignment lists, plus one extra field for additional company pages.

Field	Widget	Notes
Product Name	text input	Required
Company URL	text input	Required; must start with http(s)
Extra company pages	text area	Optional; one URL per line (About, Leadership, News)
Product Category	text input	e.g., Endpoint Detection and Response (EDR)
Competitor URLs	text area	One per line, so several competitors can be checked
Value Proposition	text area	Required; one sentence
Target Customer	text input	Required; name and title of the person being met
Product overview PDF	file uploader	Optional; PDF only

Validation. When the rep clicks Generate, the app stops with a clear error (`st.error` and `st.stop`) if the company URL does not start with `http`, or if Product Name, Value Proposition or Target Customer is empty. I tested three cases: missing fields (error shown, no brief), a URL without `https://` (error shown) and valid inputs (brief generated).

URL safety (added for resubmission). The app downloads whatever URL the rep types, and on Streamlit Cloud that download runs on the server. Without a check, someone could enter `http://localhost` or `http://192.168.1.1` and make the server read internal pages, an attack called server-side request forgery (SSRF). `is_safe_url()` now allows only `http` and `https`, looks up the site's IP address, and rejects private, loopback, link-local and reserved addresses. The same check runs before every page download, including extra pages, discovered pages and competitor sites.

Testing: `http://localhost:8501`, `http://192.168.1.1` and `http://127.0.0.1` were each stopped immediately with "That Company URL can't be used. It must be a public website." The positive control, `https://www.nymcu.org`, still produced a full brief. These cases are also automated tests (Step 11).

Step 3. Select the model and connect to it

I chose **gpt-oss-120b**, OpenAI's open-weight GPT model, through the **Groq** API.

Factor	Reasoning
Fit to assignment	The assignment asks for a GPT model; gpt-oss-120b is an OpenAI GPT model.
Cost	Groq's free tier needed no paid API key. A paid OpenAI or Anthropic key was the alternative; I chose free to keep the prototype at no cost.
Speed	Responses came back in seconds, fast enough for three chained calls per brief.
Accuracy	Followed formatting and length instructions closely in testing. Weaknesses: without data it invents plausible facts, and output varies between runs (see Step 4 and Appendix C).
Context size	Large enough to hold several web pages and a product sheet in one prompt.
Measured comparison	In the controlled experiment (Step 12), gpt-oss-120b kept product features out of the needs in both runs and wrote briefs more tailored to the prospect than gpt-oss-20b, but added more unsupported details.

I first verified the connection with a separate test script (`test_groq.py`) before adding the model to the app, so connection problems and app problems could be separated. Early tests showed the value of prompt constraints: an unconstrained question returned roughly 1,000 words, while adding length, format and audience instructions returned about 70 words in the requested format. Running the same prompt twice produced different wording and a different number of items, which showed the model is non-deterministic.

Step 4. Design the prompt and test for hallucination

The first app prompt (v1) used a four-part structure: **role** (sales research analyst), **task** (write an account brief), **context** (the form inputs) and **output rules** (length, format, headings, audience).

- **v1** produced the right format, but the model could not open the company URL. It answered from memory and invented facts, including a branch and staff count that did not add up and a case study stating that a similar credit union "reduced ransomware breach risk by 92%" for a product that does not exist.
- **v2** added a rule: use only the information provided and write "Unknown – verify" instead of guessing. The invented facts disappeared, but the brief became almost empty; the model did not even know the prospect was a credit union.

Finding: a prompt can stop a model from inventing facts, but it cannot give the model facts it does not have. The fix had to be real data.

Step 5. Integrate website data

I wrote a function, `get_page_text(url)`, that downloads a page with `requests` and extracts readable text with BeautifulSoup. Before extracting text, it removes the page's `nav`, `header`, `footer`, `script` and `style` sections, so the site menu does not crowd out the content. The first 4,000 characters of the homepage are added to the prompt.

For the resubmission I split this into `fetch_html()` (safety check, 15-second timeout, stops reading after 2 MB) and `html_to_text()` (removes menus and returns text). Keeping the raw HTML lets the app search the homepage's links for more pages (Step 6), and separating the two parts made each one testable on its own.

With real website text (v3), the brief correctly described the prospect as a credit union serving New Yorkers, listed its actual products and recent notices, and narrowed the unknowns to three specific gaps (IT staff size, current endpoint tool, compliance frameworks), which serve as discovery questions for the meeting.

Step 6. Add extra company pages for leadership and strategy

The homepage does not list leaders or most news, so I added an optional field for extra pages. For the test case I used MCU's About, Leadership Team and Community pages. Each page is downloaded with the same function inside a `try`/`except` block: if a page fails to load, the app shows a warning and continues. Every page that loads is added to the Sources list.

This step surfaced a real debugging problem: the brief reported that no leaders were named, even though the leadership page lists them. A one-line True/False test showed the names were downloaded but cut off before reaching the prompt, because each page's first 3,000 characters were mostly the repeated site menu. Stripping the menu sections in `get_page_text()` fixed every page at once; the brief went from one leader (the name I had typed in) to the full executive team. Details are in Appendix B.

A research finding from this step changed the sales approach: MCU's leadership team has no CISO, so I set the target customer to the **Chief Risk Officer, Kelly Gerber**. The talking points then shifted toward risk, compliance and audit readiness instead of technical features.

Automatic page discovery (added for resubmission)

`discover_pages()` scans the homepage's links for addresses containing about, leadership, team, executive, news, press, careers, jobs or investor. It keeps only links on the same website, skips duplicates and pages the rep already entered, and adds up to five. A checkbox turns it off, and the app shows which pages it found.

Testing: with the extra-pages field left empty, discovery found five MCU pages on its own: About, History & Mission, Leadership Team, Careers and Community. History & Mission and Careers added facts the manual run never had (15 branches, more than 700 employees, the mission statement), and the needs analysis used them as real evidence of the size of MCU's endpoint environment. **Trade-off:** discovery picked `/about-mcu/community` instead of `/about-mcu/mcu-in-the-community`, so this run lost the dated 2026 news (Chelsea branch, VaultRight, Crain's award). Discovery adds breadth, but it can miss the best page, so the manual field remains.

Step 7. Check for competitor mentions with Python

The assignment asks for "any mention of the competitors provided in the input." Because this has an exact answer, I implemented it in Python instead of asking the model. A loop takes each competitor URL,

reduces it to a name (for example `https://www.crowdstrike.com` becomes `crowdstrike`) and searches the combined text of all loaded pages, ignoring case. The results are shown on screen and passed to the model, with a rule that it may not claim the company uses a competitor unless the check found a mention.

Testing: CrowdStrike and SentinelOne were not mentioned, which is expected for a credit union website. To prove the check could also return a match, I ran a positive control with `fraud.com`; the word “fraud” appears on MCU’s site and the check correctly reported it. **Limitation:** “not mentioned on the website” does not mean “not a customer.”

Competitor website research (added for resubmission)

A name check says whether a competitor appears on the prospect’s site, but nothing about the competitor. The app now downloads each competitor’s homepage (with the same safety check) and asks the model for three bullets using only that page: what they sell, who it is for, and their main claim in their own words, writing “Not stated” when the page does not say. The summaries appear in a “Competitor research” box, each competitor site is added to Sources, and the brief adds one line per competitor on its positioning. A rule tells the model not to repeat competitor claims as facts.

Testing: CrowdStrike was summarized as the Falcon platform with the claim “We Stop Breaches with AI-native Cybersecurity”; its audience was correctly reported as “Not stated.” SentinelOne was summarized as an AI-powered platform “Built for Defenders.” The brief used this to position GreatShield as compliance-focused and sized for credit unions. The experiment in Step 12 then exposed a flaw in this feature, described there.

Step 8. Build the prompt chain

With one large prompt holding raw web text, inputs and many rules, the model dropped some rules and invented product features (for example “single pane of glass” and a subscription price). I split the work into a chain, where each step’s output becomes the next step’s input:

Step	Input	Output
1. Extract	Raw web text from all pages	Only facts stated in the text, under five headings: Company Description, Size & Members, Products & Services, Recent Initiatives, Leaders
2. Needs	Extracted facts only (never the product sheet)	3–5 likely needs, each tied to a specific fact
3. Write	Facts, needs, form inputs, product sheet, competitor results	The final account brief

Steps 1 and 2 are shown in collapsible boxes (`st.expander`), so the rep or a reviewer can check what the brief was built on. In testing, Step 1 extracted the founding year, member count, products, 20 named people and 13 dated news items. The chain removed the invented features and labeled leader motivations as “likely” instead of stating them as fact. The trade-off is three model calls instead of one, which makes generation slower.

Step 9. Parse the product overview PDF

I wrote a one-page product sheet for the fictional GreatShield EDR and added PDF parsing with `pypdf`: the app reads the text of every page, keeps the first 4,000 characters and confirms how many pages it

read. Pages without extractable text are handled safely, and an unreadable file produces a warning instead of a crash.

I compared the same test case with and without the PDF. With it, every product feature in the brief could be traced to the product sheet. Without it, small invented features returned. Grounding the product side with a document worked the same way the website did for the company side.

The PDF introduced a new problem: the model began listing product features as the customer's needs. A rule against this (v7) did not fully work, because the model could still see the product sheet while writing that section. Adding the separate needs step in Step 8 (v8), which never sees the product sheet, fixed it.

Step 10. Produce the one-pager and download

The brief uses headings that match the assignment's required outputs. A title block (product, meeting, company, date) is added, and the source list is built by Python so the links are always real. A download button saves the one-pager as a Markdown file; setting `on_click="ignore"` keeps the brief on screen after downloading.

Assignment output	How the one-pager covers it
Company Strategy	Company Strategy section, from extracted facts and dated news (e.g., new branch, VaultRight selection)
Competitor Mentions	Competitor Mentions section, from the Python check
Leadership Information	Leadership section with names, titles and likely relevance
Product/Strategy Summary	Covered from website facts. MCU is a member-owned credit union, not a public company, so there is no 10-K to use.
Article Links	Sources section lists every page used. Links to individual news articles are a planned improvement.

A sample one-pager from the live app is in Appendix F.

Step 11. Refactor the code and add automated tests

Tests cannot import the original `app.py`, because importing it builds the Streamlit page and connects to Groq with an API key. I moved every function into a new file, `agent_core.py`, which has no Streamlit code and needs no key. `app.py` now only draws the form, calls those functions and shows the results. The prompts were copied word for word, which I confirmed automatically.

`tests/test_agent_core.py` holds 42 pytest tests. Where the real code would go online (DNS lookups, web pages, the model), the tests swap in fakes with pytest's `monkeypatch`, so they run in under a second with no internet or API key, and anyone can run them with `uv run pytest`.

Area from the review	What the tests check
Required fields	Valid inputs pass; each missing required field returns the error message
URL validation	Non-http schemes, missing host, unknown domains, and 127.0.0.1, 192.168.1.1, 10.0.0.5 and 169.254.169.254 are rejected; public addresses are allowed
Page extraction	Menus, headers, footers and scripts are removed; pages over 2 MB are refused
Failed requests	An unsafe URL is never downloaded; network timeouts raise an error

Area from the review	What the tests check
Competitor checks	Names are taken from URLs; mentioned and not-mentioned cases; blank lines ignored
PDF parsing	Text is read from a PDF; long text is cut to the limit; a broken file raises an error
Prompt construction	Each prompt contains its inputs and key rules; the needs prompt never contains the product sheet
Source preservation	Every source URL is kept in order; the one-pager contains the header, brief and sources
LLM failure handling	Model answers are returned; connection errors and empty answers become a friendly error
Discovery and competitor research	Same-site links only, duplicates and entered pages skipped, page limit respected; competitor prompt uses only competitor text

Proving the tests work: I deliberately removed `ip.is_private or` from the safety check and re-ran the tests. Two failed (192.168.1.1 and 10.0.0.5 now got through); 127.0.0.1 and 169.254.169.254 were still caught by the loopback and link-local checks, which showed exactly what each part of the line protects. After undoing the change, all tests passed.

Bugs fixed by the refactor: a blank line in the competitor box used to report ": mentioned on the prospect's website" (an empty name matches any text); a failed main URL crashed the app with a traceback instead of an error message; and an empty model answer produced a blank brief instead of an error.

Step 12. Run a controlled experiment

To test two design decisions with evidence instead of impressions, I wrote `experiments/run_experiment.py`. It downloads the standard test case's pages once and saves them to `evidence.json`, so every run reads identical text, the same product sheet and the same rules. Only one variable changes per comparison, and each condition runs twice because output varies between runs.

Condition	Prompt design	Model
A	3-step chain (the app's design)	gpt-oss-120b
B	Single prompt with all raw text	gpt-oss-120b
C	3-step chain	gpt-oss-20b (Llama 3.3 70B was planned but is not available on my Groq account)

The script scores each brief automatically (time, word count, headings, real leaders named, invented names, product-sheet words under Likely Needs). I then read every brief against the evidence and counted unsupported claims by hand. Full results are in Appendix C and in `experiments/results/`.

Run	1st-run time	Product words in Likely Needs	Unsupported claims (manual)
A – chain, 120b	7.7 s	none, none	3, 5

Run	1st-run time	Product words in Likely Needs	Unsupported claims (manual)
B – single prompt, 120b	19.0 s	4 and 5 (all 6 needs were product features)	2, 3
C – chain, 20b	6.2 s	none, 1 (NCUA)	2, 1

- **Chain vs. single prompt:** the chain did the job it was built for. In both single-prompt runs every "likely need" was a GreatShield feature (rollback, isolation, triage), and product-sheet claims leaked into Company Strategy; in both chain runs, none did. The chain did not stop speculation, though: chain runs still added details the pages never mention (for example ATMs and POS terminals).
- **A flaw found in my own feature:** four of the eight unsupported claims in the 120b chain runs were invented competitor positioning. The brief prompt asked for "one line per competitor on its positioning from the competitor research," and in the experiment there was no research, so the model filled the gap from memory. I changed the prompt to describe a competitor only when the research does. In the app this happens only if a competitor site fails to load or research is turned off, but it was a real hallucination path.
- **120b vs. 20b:** the smaller model was slightly faster and more cautious, but leaked a product term into needs once and wrote generic feature-list talking points. gpt-oss-120b wrote briefs tailored to the prospect. With two runs each this is a trade-off to watch, not a proven ranking.
- **Measurement lessons:** my first scoring reported zero leaders in two briefs; checking the files showed the model had used an invisible narrow no-break space between first and last names, so the name search failed. I fixed the scorer and re-ran. Every second run took 34–40 seconds because Groq's free tier slows down after a burst of calls, so only first-run times are compared.

Step 13. Deploy and plan for production

Deployment. I generated `requirements.txt` with `uv export`, pushed the code to a public GitHub repository and deployed it on Streamlit Community Cloud. The Groq API key is stored in Streamlit's Secrets settings, not in the code; Streamlit exposes it as an environment variable, so no code changes were needed. A full test on the live URL (standard test case with PDF) generated the brief and the download successfully. Pushing to GitHub triggers an automatic redeploy.

Production deployment plan

The prototype runs as one public Streamlit app on a free model quota. A production version used by a sales team would need the following.

Area	Current prototype	Production plan
Authentication	Public link; anyone can run it	Company single sign-on (e.g., Microsoft Entra ID or Okta); only sales staff can use the app
HTTPS	Streamlit Cloud serves HTTPS	Host behind HTTPS with a company domain and certificate; redirect all HTTP
Rate limiting	None; every run uses my free Groq quota	Per-user limits (e.g., 20 briefs per hour) and a global limit; clear message when a limit is hit
Caching	Every run downloads every page again	Cache page text and extracted facts per URL for 24 hours (e.g., `st.cache_data` or Redis), so repeat briefs are fast and cheaper

Area	Current prototype	Production plan
Queues	Brief is built while the user waits (5 model calls)	Put brief requests on a background job queue (e.g., Celery or a cloud queue) and notify the user when ready
Monitoring	None beyond on-screen errors	Track response time, model errors, rate-limit hits and failed page loads on a dashboard, with alerts when error rates rise
Logs	None	Structured logs of each run (time, URLs, model, errors) without storing API keys or personal data; logs kept 30 days
Cost budgets	Free tier	Paid model tier with a monthly budget and spend alerts; estimate cost per brief; use a smaller model for extraction to save cost
CI/CD	Push to GitHub redeploys the app; tests run by hand	GitHub Actions runs `pytest` on every push and blocks deployment if any test fails; deploy to staging first
Rollback	Revert a commit and push	Tagged releases; one-click redeploy of the last good version; prompt versions tracked so a bad prompt can be reverted alone
Data retention	Nothing stored; briefs exist only in the browser and the download	If briefs are saved, keep them for a set period (e.g., 90 days), delete on request, and never store uploaded product sheets longer than needed
Multi-user isolation	Each Streamlit session is separate, nothing shared	Saved briefs and uploads tied to the user who created them; no user can see another's data
Security: user URLs	**Implemented:** only http/https; private, loopback, link-local and reserved IPs blocked; 2 MB limit; 15 s timeout	Also re-check the address after redirects, and limit how many pages one request can fetch
Security: web content	Web text is placed in the prompt as data	Treat fetched text as untrusted (it could contain instructions aimed at the model); keep it clearly delimited and never let it change the system rules
Maintenance	42 automated tests; versions pinned; prompts in Git	Re-run the tests and the experiment after every prompt or model change; watch for model deprecation; schedule dependency updates

Step 14. Optional enhancements

Email alert system (designed, not built)

The alert system would watch each prospect for new content containing the rep's keywords and send one daily digest email.

Flow: daily schedule → fetch watched pages → find new links → keyword match (Python) → one-line AI summary → email digest

- **Inputs:** watched company URLs (news and careers pages), keywords such as “cybersecurity,” “security engineer” or “breach,” the rep’s email and frequency.
- **Schedule:** a separate scheduled job (for example a GitHub Actions scheduled workflow), because a Streamlit app only runs while someone has it open.
- **Change detection:** the job saves the links it has already seen and only processes new ones, so the same alert is never sent twice.
- **Matching:** Python keyword search first (exact, cheap, cannot hallucinate); the model only writes a one-sentence summary for matched items.
- **Notification:** one digest email per day with title, date, link, matched keyword and summary; no email on days with no matches; email credentials kept in secrets.
- **Safeguards:** respect sites’ automated-access rules, one request per page per day, an unsubscribe option, and the original link always included so AI summaries can be checked.

Improving output quality

Already implemented: grounding in real data, a three-step chain, Python for exact tasks, explicit rules, automated tests and a controlled experiment. Further improvements: lower the model temperature to reduce run-to-run variation, require each bullet to cite its source page, add a verification step that checks the brief against the extracted facts, and return structured output (JSON) so each section can be checked automatically.

Using other types of models

- **Meeting deck:** pass the finished brief to a model with a slide outline prompt, then build slides with a presentation library.
- **Documents the rep shares** (RFPs, call notes): split them into chunks, store embeddings, and retrieve the relevant parts for each question (retrieval-augmented generation).
- **Model mix:** a small, fast model for fact extraction and a larger model for the final writing, to balance cost and quality. The experiment suggests this could work: gpt-oss-20b was faster and cautious, while gpt-oss-120b wrote better final briefs.
- **Scanned PDFs or images:** an OCR or vision model to read documents that have no extractable text.

Conclusion

The capstone produced a deployed sales assistant agent that turns public web pages and a product sheet into a sourced one-page account brief. The most valuable part of the project was the prompt engineering process: eight versions tested against one fixed case, each changing one thing, with the results recorded. The resubmission added automated tests, SSRF protection, competitor research, page discovery and a controlled experiment.

Key takeaways

1. **Ground the model in real data.** Rules reduced invented facts, but only website text and the product sheet added true facts.
2. **Structure beats more rules.** When the model kept turning product features into customer needs, a separate step that could not see the product sheet fixed what an explicit rule could not.
3. **Give code the exact jobs.** Competitor checks, source links, validation and page cleaning are done in Python, which cannot hallucinate and returns the same answer every time.
4. **Test the cause before changing code.** A one-line True/False check separated “not downloaded” from “downloaded but cut off,” which needed different fixes.
5. **Measure, then check the measurement.** The controlled experiment confirmed the chain’s main benefit, exposed a hallucination path in a feature I had just added, and showed that an automatic score can be wrong (the invisible space) until it is checked against the real output.

Limitations

- The model still makes small, varying assumptions between runs (for example devices or preferences not stated in the facts); the experiment found 1–5 unsupported claims per brief, so a human must review each brief.
- Automatic page discovery can pick a weaker page than the one a person would choose, and is limited to five pages.
- The experiment used two runs per condition and one test case; its findings are consistent but not statistically proven.
- Product features occasionally appear in the Leadership section.
- The competitor check only searches pages the rep provides, and an absent mention does not prove the company is not a customer.
- Sources are listed per page, not per news article.
- Pages that build their content with JavaScript may return little text.

Next steps

Run the tests automatically on every push with GitHub Actions; add per-article links and per-bullet citations; lower the temperature; add login and rate limits to the live app; add a third model from a different vendor (for example Claude through the Anthropic API, which requires paid credits) to the experiment; and build the alert system described in Step 14.

Appendix

A. Time Management

Times are approximate, taken from my session logs.

Date	Hours	Focus
Oct 7–10	~1.5	Reviewed assignment and rubric; chose model; set up project; built input form
Oct 11	~2.5	Finished form; connected Groq; first prompt experiments (constraints, audience, repeat runs)
Oct 14	~7	Core build: model in app, website data, competitor check, extra pages, validation, prompt chaining, PDF parsing; prompts v1–v7
Oct 15	~2	Needs-analysis step (v8) and testing
Oct 16	~4	One-pager download, error handling, GitHub, deployment and live testing, alert design, report
Oct 22	~5	Resubmission: docs and README, SSRF protection, refactor and 42 tests, competitor research, page discovery, controlled experiment, report update
Total	**~22**	

Task area	Share	Why
Data integration (web pages, discovery, competitor research, PDF)	~25%	The model was only as good as its inputs; this had the biggest effect on accuracy
Prompt engineering and experiments (v1–v8, controlled experiment)	~25%	The core skill of the course; each version was tested against the same case
Testing, security and debugging	~20%	42 automated tests, SSRF protection, API key and VPN errors, missing leadership data
Interface, validation and output	~10%	Required inputs, error handling, one-pager and download
Setup and deployment	~10%	Environment, Git, GitHub, Streamlit Cloud
Documentation	~10%	Notes kept during every session made this report faster to write

Approach: I built the smallest working version first (form → model → output) and then added one feature at a time, testing and committing after each. I kept a running notes file with timestamps, challenges and experiment results during every session.

B. Challenges and Solutions

Challenge	Cause	Solution
IndentationError / SyntaxError in early code	Misplaced spaces, quotes and parentheses	Learned Python's indentation and function-call structure; fixed line by line

Challenge	Cause	Solution
Edits not taking effect	File not saved before running	Turned on VS Code Auto Save
Groq 403 “Access denied – check network settings” (Oct 11 and Oct 16)	VPN was on; Groq blocked the VPN’s network	Turned the VPN off; wrapped model calls in try/except with a clear error message
Groq 401 “Invalid API Key”	The .env file had been changed by accident	Generated a new key; the error code separated key problems (401) from network problems (403)
Prompt broke with SyntaxError	Double quotes inside a double-quoted f-string	Used single quotes inside the prompt
Hallucinated facts and fake case study (v1)	Model cannot open URLs; answered from memory	Anti-hallucination rule (v2), then real website text (v3)
Wrong test inputs	Used a security vendor as the prospect	Prospect in Company URL, vendor in Competitors
Leadership names missing	3,000-character slice held only the site menu	Diagnosed with a True/False test; removed nav/header/footer before extracting text
Test data appeared in the brief	Positive-control value left in the competitor list	Removed test values before real runs
Invented product features	One large prompt; rules dropped	Prompt chaining (extract, then write)
Product features listed as customer needs	Model could see the product sheet while writing needs	Separate needs step that never sees the product sheet
One-pager formatting (merged lines, two titles)	Markdown ignores single line breaks; model added its own title	Bullet-list title block; rule to start directly with heading 1
Report and deck not visible to the grader	Files were kept on my PC, not in the repository	Added a `docs/` folder with the report, presentation and sample output
Files saved as `.md.md` and `.pdf.pdf`	Windows hides file extensions, so I typed the extension twice	Renamed with `git mv`; turned on File name extensions in Explorer
App code could not be tested	Importing `app.py` builds the page and needs an API key	Moved logic into `agent_core.py`; tests use fakes instead of the internet
Updated file reverted to an old version	VS Code had the file open and saved its older copy over the new one	Close a file’s tab before it is replaced, or reopen it afterwards
Experiment crashed reading the PDF	The PDF in the folder was a text file renamed to .pdf (691 bytes)	Copied the real PDF; the error “invalid pdf header” pointed to the cause
Experiment reported 0 leaders in two briefs	Model used a narrow no-break space inside names, so the search failed	Normalized special spaces before scoring; re-ran the experiment
Llama 3.3 70B returned 404	Model not available on my Groq account	Script fell back to gpt-oss-20b; reported as a size comparison
Invented competitor positioning	Prompt asked for positioning even when no research existed	Prompt now describes a competitor only if the research does

C. Experiments Log

All app experiments used the standard test case in Appendix G, changing one thing at a time.

Ver.	Change	Result
Baseline	Unconstrained question in test script	~1,000 words, tables, off-target
Constraints	Added length, format, audience	~70 words in the requested format; repeat runs varied in wording and item count
v1	Four-part prompt (role, task, context, output rules)	Correct format; invented company size, case study and product features; no sources
v2	+ “use only the information provided / Unknown – verify”	Invented facts gone; almost no useful facts
v3	+ homepage text (requests + BeautifulSoup)	Grounded facts; only three specific unknowns; one misread page
v4	+ Python competitor results; sources listed by Python	Accurate competitor section; false “all facts from website” claim removed
v5	+ extra pages; assignment headings; menu stripping	Real executive team and dated news; invented product features remained
v6	Prompt chaining: extract facts, then write	Invented features gone; leader roles labeled “likely”; 7 leaders
v7	+ product PDF; three tighter rules	Real features from PDF; three issues fixed; needs still restated product features
v8	+ separate needs step (company facts only)	Needs tied to company facts; product features only in talking points
PDF test	Same case with vs. without PDF	With PDF: no invented features; without: some returned
Positive control	fraud.com as a competitor	Check correctly reported a match
v9	+ competitor website research; automatic page discovery	Competitor positioning in their own words; 5 pages found automatically with new facts (15 branches, 700+ employees); dated news page missed
v9.1	Competitor positioning only when research exists	Fix for the hallucination path found by the controlled experiment

Controlled experiment (resubmission)

Same saved evidence for every run (`experiments/evidence.json`), standard test case, two runs per condition. Unsupported claims were counted by hand against the evidence; product features listed as needs are counted separately.

Run	Time	Words	Real leaders	Invented names	Product words in Needs	Unsupported claims
A1 chain 120b	7.7 s	408	5	none	none	3
A2 chain 120b	40.2 s*	539	5	none	none	5

Run	Time	Words	Real leaders	Invented names	Product words in Needs	Unsupported claims
B1 single 120b	19.0 s	403	5	none	agent, isolation, ncua, triage	2
B2 single 120b	18.0 s	442	5	none	agent, isolation, ncua, rollback, triage	3
C1 chain 20b	6.2 s	344	5	none	none	2
C2 chain 20b	34.2 s*	317	5	none	ncua	1

*Second runs were slowed by Groq free-tier throttling. Examples of unsupported claims: invented CrowdStrike and SentinelOne positioning (A1, A2), “ATMs and POS terminals” at the new branch (A1), staff “work from multiple sites” (A2), NCUA/FFIEC compliance listed as an MCU priority (B1), “MCU’s small security staff” (B1, C1, C2), “grants to St. Francis College” when it was a partnership (C1). The full list is in `experiments/results/summary.md`.

D. Final Prompts

Step 1 – Extract

```
Read the company web page text below and list ONLY facts that are directly
stated in it. Group them under these headings: Company Description, Size &
Members, Products & Services, Recent Initiatives, Leaders (name and exact title).
Use short bullet points. Do not interpret, guess, or add anything that is not in
the text. If a heading has nothing, write 'None found'.

Homepage text: {website_text}
Other company pages: {extra_text}
```

Step 2 – Needs

```
You are a business analyst. Based ONLY on the company facts below, list 3 to 5
likely needs this company may have related to {product_category}.
For each need, give one short reason tied to a specific fact from the list.
Do not mention any vendor, product, or product feature. If the facts don't
support a need, leave it out.

Company facts: {facts}
```

Step 3 – Write

```
You are a sales research analyst helping a sales rep prepare for a first meeting.
Write a one-page account brief about the target company.
The rep sells {product_name}, a {product_category} product. Its value:
{value_proposition}. The target company's website is {company_url}.
The rep will meet with {target_customer}.

Use these headings, with short bullet points under each:
1. Company Strategy  2. Leadership
3. Competitor Mentions: check results; one line on positioning ONLY if the
   competitor research describes it (if None, describe no competitor)
4. Likely Needs (needs analysis only, keeping each reason)  5. Talking Points

Rules:
- Keep it under 350 words. Write for a rep with 10 minutes to prepare.
- Use only the verified facts below. If something is missing, write
  'Unknown - verify'.
- Describe the product using ONLY the value proposition and product overview.
- Do not state what a leader decides, owns, or prefers unless the facts say so;
  every statement about a leader's role or interest must include 'likely'.
- Do not invent names, titles, statistics, or case studies.
- Do not claim the company uses a competitor unless the results say it is
  mentioned; an absent mention is never an opening or opportunity.
- Use competitor research only to describe what each competitor says about
  itself and how the rep can differentiate; never repeat its claims as facts.
- Company Strategy and Likely Needs come only from company facts.
- Do not say where facts came from; sources are listed separately.
- Start directly with heading 1. No title, meeting line, or closing note.

Needs analysis: {needs}   Product overview: {product_text}
Competitor check results: {competitor_results}
Competitor research: {competitor_research or 'None'}   Verified facts: {facts}
```

Competitor research (one call per competitor)

```
Below is text from the website of {name}, a competitor.
Using ONLY this text, write 3 short bullet points:
- What they sell (main product or platform)
- Who they say it is for
- Their main claim or differentiator, in their own words
Do not add anything that is not in the text. If something is not stated,
write 'Not stated'.

Website text: {page_text}
```

E. Code Structure

The code is split so the logic can be tested without the interface. `app.py` only handles the Streamlit page; every function is in `agent_core.py`.

Function (agent_core.py)	What it does
`is_safe_url(url)`	Allows only http/https URLs that resolve to a public IP address

Function (agent_core.py)	What it does
`validate_inputs(...)`	Returns an error message for a bad URL or empty required field, or None
`fetch_html(url)` / `html_to_text(html)`	Safe download (15 s timeout, 2 MB limit) / readable text without menus and scripts
`discover_pages(base_url, html, ...)`	Finds up to five same-site About, Leadership, News, Careers and Investor links
`check_competitors(urls, text)`	Mentioned / not mentioned for each competitor
`build_competitor_prompt(...)`	Prompt that summarizes a competitor from its own homepage
`read_pdf_text(file)`	Text and page count from the product PDF
`call_model(client, prompt, model)`	Calls the model; raises a friendly ModelError on failure or an empty answer
`build_extract_prompt` / `build_needs_prompt` / `build_brief_prompt`	The three prompts of the chain
`build_sources_text` / `build_one_pager`	Sources list and downloadable Markdown one-pager

Repository files: `app.py` (Streamlit page), `agent_core.py` (logic), `tests/test_agent_core.py` (42 pytest tests), `experiments/` (experiment script, saved evidence, results), `docs/` (this report, the presentation, a sample one-pager), `README.md`, `requirements.txt` (deployment dependencies), `pyproject.toml` and `uv.lock` (local environment and pytest settings), `notes.md` (session log), `test_groq.py` and `test_fetch.py` (early connection tests), `.gitignore` (excludes `.env`).

F. System Output: Sample One-Pager

Generated on the live app, October 16, 2026, with the standard test case and product PDF. Lightly reformatted for this report; content unchanged.

Account Brief

- **Product:** GreatShield EDR
- **Meeting with:** Kelly Gerber, Chief Risk Officer
- **Company:** https://www.nymcu.org
- **Date:** October 16, 2026

1. Company Strategy

- Serves > 630,000 NYC members with personal, home, auto, and credit-card products, plus budgeting tools and online loan services.
- Mission: “empowering hard-working New Yorkers” and “helping the people who power this city.”
- Recent focus on expanding physical presence – new Chelsea branch opened Aug 31 2026.
- Strengthening branch operations through technology – selected VaultRight Aug 20 2026.
- Emphasizing community impact and employee engagement – Best Places to Work award Oct 2 2026; multiple foundation partnerships (Bowery Mission, St. Francis College, Forestdale, Children’s Aid).

2. Leadership

- **Kelly Gerber – Chief Risk Officer** – likely focused on risk mitigation, regulatory compliance, and protecting member data from endpoint threats.

- **Jim Mill – Chief Technology Officer** – likely interested in scalable, low-maintenance security solutions that fit existing tech stack.
- **Kyle Markland – Chief Executive Officer** – likely cares about overall risk posture and reputation as a trusted community bank.
- **Frank Madeira – Chief Financial Officer** – likely evaluates cost-effective solutions that don't require large security staffing.
- **Toni-Ann Sforza – Chief Operating Officer** – likely looks for tools that support smooth branch operations and remote-work continuity.

3. Competitor Mentions

- crowdstrike – not mentioned on the prospect's website.
- sentinelone – not mentioned on the prospect's website.

4. Likely Needs

- **Real-time monitoring of all workstations and laptops** – MCU serves > 630,000 members; a breach on any device could expose sensitive personal and financial data.
- **Rapid automated containment of ransomware or malware attacks** – CRO role highlights risk management; ransomware could halt loan processing, online payments, and budgeting tools.
- **Visibility and protection for branch-level devices** – New Chelsea branch and VaultRight selection indicate need to safeguard PCs, POS systems, and printers at branch sites.
- **Endpoint forensics and incident-response reporting for audit & compliance** – As a credit union handling regulated products, detailed logs support regulator-required audit trails.
- **Support for remote-work and mobile device security** – Online loan applications, member portals, and budgeting apps expand the attack surface.

5. Talking Points

- GreatShield EDR delivers enterprise-grade endpoint protection designed for mid-sized financial institutions without large security teams.
- Continuous monitoring across Windows, macOS, and Linux workstations/servers matches MCU's need for real-time visibility.
- One-click device isolation and ransomware rollback provide rapid containment and recovery.
- Audit-ready compliance reports (NCUA/FFIEC mapped, exportable PDF) support the CRO's regulatory reporting requirements.
- Managed 24/7 alert triage means MCU's limited security staff receive only confirmed threats.
- Cloud-managed, lightweight agents deploy through existing software tools.

Sources

- https://www.nymcu.org
- https://www.nymcu.org/about-mcu/
- https://www.nymcu.org/about-mcu/leadership-team
- https://www.nymcu.org/about-mcu/mcu-in-the-community

Review of this output: facts, leaders and dates trace to the source pages and product sheet. Remaining issues: "community bank" (MCU is a credit union), "POS systems, and printers" (not in the source facts), and "CTO's preference" stated without evidence in one run. Leadership names and titles were checked against the leadership page: 5/5 correct. A later run, saved in the repository as `docs/sample_account_brief.md`, listed six leaders: 6/6 correct.

G. Test Case, Sources and AI Assistance

Standard test case

Field	Value
Product Name	GreatShield EDR (fictional)

Field	Value
Company URL	https://www.nymcu.org
Extra pages	/about-mcu/ · /about-mcu/leadership-team · /about-mcu/mcu-in-the-community
Product Category	Endpoint Detection and Response (EDR)
Competitors	https://www.crowdstrike.com · https://www.sentinelone.com
Value Proposition	Enterprise-grade endpoint protection with audit-ready compliance reporting, built for mid-sized financial institutions without large security teams.
Target Customer	Kelly Gerber, Chief Risk Officer
Product PDF	GreatShield_Product_Overview.pdf (1 page, written by me)

Links

- Live app: https://cap931-sales-agent-rwjr.streamlit.app/
- Code repository: https://github.com/rwnycvet/cap931-sales-agent
- Prospect sources: https://www.nymcu.org and the three pages listed above
- In the repository: `docs/` (report, presentation, sample one-pager), `experiments/results/` (experiment briefs and summary)

AI assistance

I used Claude (Anthropic) as an instructor throughout the project for explanations, debugging guidance and code patterns. Several code blocks, the first app prompt, the alert-system design, the refactor into `agent_core.py`, the test suite, the experiment script and drafts of this report were produced with AI assistance; I implemented, ran and tested every version of the app, recorded the results, and reviewed and edited this report. The app itself uses gpt-oss-120b through the Groq API.